Logic is the Beginning of Wisdom

King-Yin Yan[0009−0007−8238−2442]

general.intelligence@gmail.com

Abstract. ... not the end of it; so said Spock in a 1991 Star Trek movie. This paper introduces Logic Transformer, a differentiable neural network that explicitly performs classical symbolic logic inference (with rules matching, substitutions, etc). Preliminary experiments show that it achieves a very high compression ratio (ie fewer parameters) and also seems to train faster as a consequence. From the "No Free Lunch" perspective this suggests that algebraic/categorical structure of logic is suitable for compressing our world (as Wittgenstein says, logic is a mirror-image of the world). A high compression ratio forces the knowledge contained in our neural network to attain "logic coherence", which we believe is the antidote to hallucinations. The explicit logic structure also opens the "black boxes" of neural networks for inspection and interpretation. Moreover, it allows to use a classical symbolic logic inference engine (eg. OpenCog, OpenNARS) side-by-side with our neural network in a hybrid architecture, that may offer advantages we are only beginning to explore. For all these implications, we claim that this is a key innovation that solves the AGI problem.

Keywords: logic-based AI · neural-symbolic integration · LLMs (large language models)

1 Background: RL + LLM Architecture

This section outlines the basic framework required for subsequent work. We aim to combine RL (reinforcement learning) and LLMs (fig. 1). LLMs are traditionally implemented by Transformers, to be replaced by Logic Transformers.

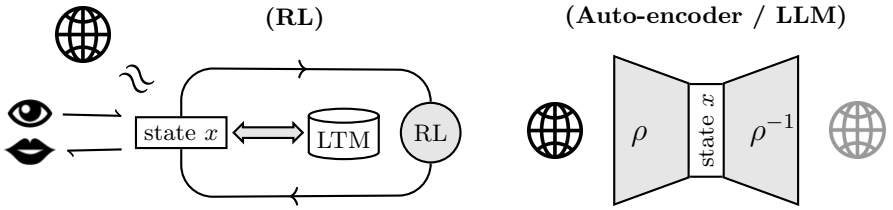


Fig. 1. Left: basic RL setup; LTM = long term memory. Right: The essence of an auto-encoder, of which LLMs are a special case, is to compress (via ρ) and reconstruct (via ρ^{-1}) world data.

Two types of architectures are possible (fig. 1), where a traditional LLM may serve as a world-model (**W**) or as a model of thinking processes (**T**). It turns out the latter is more natural and similar to mainstream approaches such as RLHF [**Li2023**]. It implies that "prompts" are regarded as Working Memory and offers the tremendous advantage that humans can read the thoughts of an AGI directly in natural language.

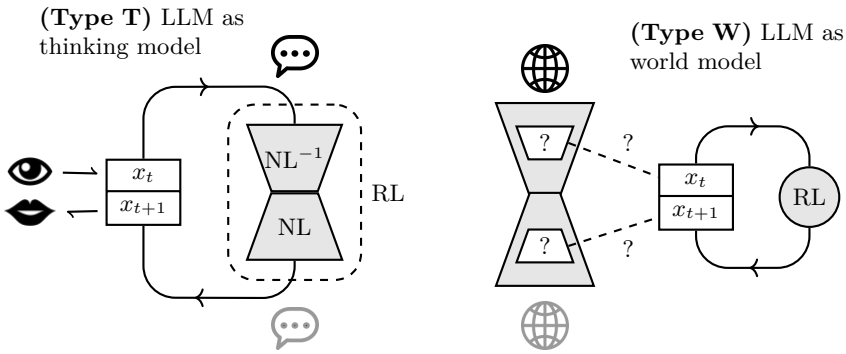
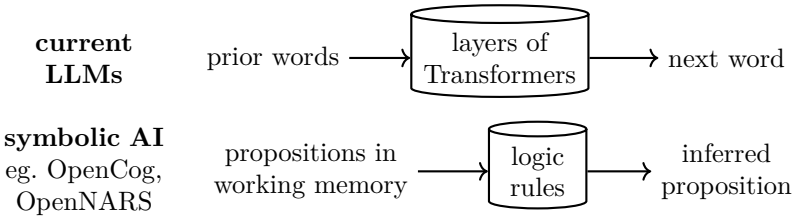


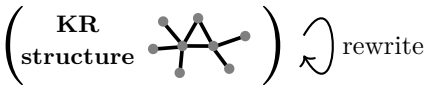
Fig. 2. Two types of RL + LLM architectures. NL^{-1} is a map that compresses natural language to a hidden representation (not shown explicitly).

As we know, LLMs predict the next word per each iteration. For a long time I have been trying to reconcile the following two views:



until I had the insight that a logic-based system could output a proposition to rewrite part of a linguistic tree or graph, similar to the LLM outputting the next word to complete a sentence.

This leads us to the following basic architecture, which is also the setup proposed by Goertzel in [**Goertzel2021**]:



(1)

One way to define a rewrite rule [**Kennaway1993**] is as a pair of morphisms $L \leftarrow K \rightarrow R$, L and R are the left and right hand sides, and K is the interface

graph. G is the input graph and H is the result of rewriting. The effect is that the part of L outside K is deleted from G and is replaced by the part of R outside K:

$$\begin{tikzcd} L \arrow[d] & K \arrow[l] \arrow[r] \arrow[d] & R \arrow[d] \\ G & D \arrow[l] \arrow[r] & H \end{tikzcd} \tag{2}$$

This occurs in a category where graphs are objects and homomorphisms define some notion of "matching". Now if we imagine a **differentiable** version of rewriting rules (see (4) below [1]), all the rules would be represented by a set of parameters $\Theta \in \mathbb{R}^n$, and the parameterized rules *must* cover the entire family of rewrite rules as Θ vary continuously (in practice, we can limit the rules length to make that possible). This requires the use of "**discretizing**" functions such as sigmoid and softmax, and may be formulated by a fairly mechanical process. However, in doing so we have overlooked the possibility that a **deep** neural network such as Transformer may be able to aggregate all the contents of Working Memory, output some intermediate **distributive** representations, before finally outputting some discrete tokens. Such a process would imply L matches the entire input graph G in (2).

2 Logic Transformer

In classical logic-based AI, a single step of inference is a mapping $\Delta \overset{\text{KB}}{\longmapsto} \Delta + \delta$ where KB is a set of logic **rules** and Δ is a set of ground propositions or **facts**. Δ also called the state $\boldsymbol{x}$ or Working Memory. "Set" implies algebraic structure, ie, permutation invariance of elements. We recognized early on that this has a connection with the equivariance structure of Self-Attention; but it was not obvious how to extract further algebraic structures from logic and to apply such symmetries; instead, we found a way to actually implement the syntactic process of substitutions and logic-rule applications, so the resulting neural network obviously inherits logic structure implicitly. A logic rule is a formula with variables such as:

$$\forall X\ \forall Y\ \forall Z. \quad \text{father}(X, Y) \wedge \text{father}(Y, Z) \Rightarrow \text{grandfather}(X, Z) \tag{3}$$

where the variables must obey strict discipline (X matches to X, etc) as shown by dotted lines.

A key innovation is what I termed the **cylindrification factor** γ, which allows a logic constant to change continuously into a variable, as illustrated in fig. 3.

Furthering the idea, the matching (or **unification**) of variables makes use of a fixed number of variable "slots" where we "move" contents in and out of the

[1] with experimental code available on https://github.com/Cybernetic1/reinforcement-learning-experiments

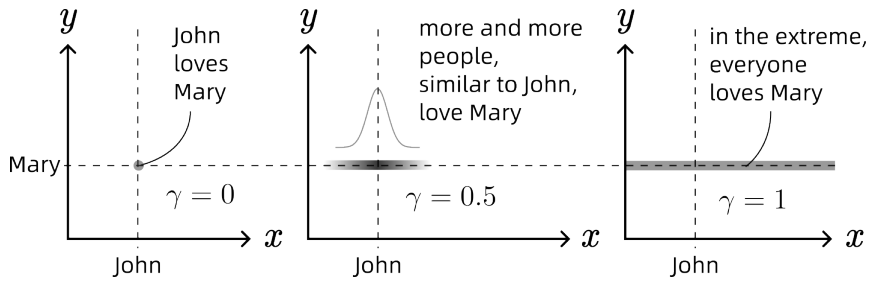
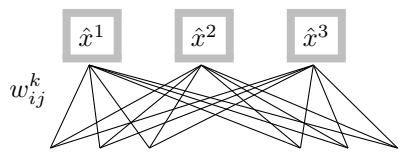


Fig. 3. Illustration of the cylindrification factor γ.

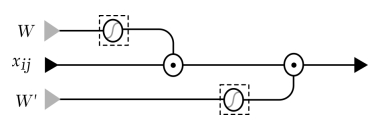
slots by neural weights with **Softmax**, so that the transfer of contents can vary continuously:



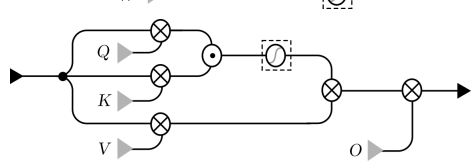
$$\mathsf{father}(x_{11}, x_{12}, \bullet_{13}) \wedge \mathsf{father}(\bullet_{21}, x_{22}, x_{23}) \rightarrow \mathsf{grand\text{-}father}(x_1, \bullet_2, x_3) \tag{4}$$

which has a form quite similar to the **Self-Attention** of Transformers (right):

$$\hat{x}^k := \underset{\substack{i \in \text{predicates} \\ j \in \text{arguments}}}{\forall} \left\langle \operatorname*{soft\,max}_{ij} w^k_{ij} \ , \ x_{ij} \right\rangle$$



$$\alpha(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{soft max} \frac{\langle \mathbf{Q}, \mathbf{K} \rangle}{\sqrt{d_k}} \mathbf{V}$$



(5)

but the rule (4) neglected the fact that the variables in each father atom must match their variables with each other (a situation well-known in classical logic-based AI). This means we have to programmatically feed all *pairs* of Working-Memory atoms, ie. the Cartesian product $A \times A$, into the above string diagram. That makes it different from the Transformer which takes its input in plain form. The reason why Transformer can do that is because it has many **layers**, and the matching of $A \times A$ can be achieved over these layers.

Indeed, neural **associative memory** (such as Transformer, which are shown to be equivalent to Hopfield Networks [**Ramsauer2021**]) may be a far superior form of "logic" than traditional, formal symbolic logic. Consider the logic example:

$$\text{loves}(X, Y) \wedge \neg\text{loves}(Y, X) \rightarrow ☹(X) \tag{6}$$

but we immediately sense that the human brain does not work like that. It is more like: the thought of unrequited love fires a massive number of neural

activations which all point to the conclusion that this is an unhappy experience. The Transformer's associative memory does exactly that.

Also recall that Self-Attention originated as a kind of **content-addressable memory** for Neural Turing Machines, invented for its differentiability. In retrospect, it is also a very efficient implementation of associative memory. If we want to improve the Transformer, we should first understand it as an associative memory.

From a **functorial** point of view (eg.[**Crescenzi2024**] [**Abbott2024**] [**Khatri2024**]), one can find a functor that transforms a category $\mathcal{C}$ of KR structure and rewrite morphisms into another category $\mathcal{D}$ with different KR structure and rewrite morphisms. It might just happen that one type of rewrite morphisms are computationally less costly than the other type, and so we have discovered a better, more efficient AGI system. But this maneuver seems not so useful in light of the fact that Transformer / Self Attention is a distributive associative memory that may not care too much about its input representation.

3 Additional Features and Enhancements

Davidsonian grammar.
Reflection.
Reification.
HOL.
Horn form.
Vector Quantization.

4 Conclusion

We proposed a simple AGI architecture that looks promising. If the Transformer cannot be improved much further, we should perhaps accept the current design and proceed to the "engineering" phase of AGI development.